



**Lahore Garrison University**

**Project Proposal : Horizon: A Case Study in Software  
Construction for a Web-Based Streaming Platform**

***Group Members:***

Abeeha Raza/094/C

Alina Faisal/095/C

Haziq Zaman/104/C

Salman Rao/091/C

## 1. Process Model Implemented and Its Justification

For the development of *Horizon*, we adopted the Agile Software Development Model, specifically leveraging Scrum. The project was divided into multiple sprints, each focusing on iterative improvements, modular component development, and feature integration such as search, routing, theming, and responsive design. Agile was selected due to the dynamic nature of front-end development and our use of React, where features are broken into independent, reusable components. This allowed us to respond quickly to changes in UI/UX direction, prioritize component-level testing, and refactor efficiently based on team feedback. Frequent sprint reviews ensured that the application aligned with user expectations and software construction principles.

## 2. Software Process Improvement (SPI) During the Semester Project

Yes, Software Process Improvement (SPI) was actively practiced during the project lifecycle. Early in the development phase, we identified inefficiencies such as inline styling, duplicated logic, and lack of standard theming. In response, we transitioned from vanilla HTML/CSS/JS to React with Material UI, which provided consistent theming, responsive grids, and scalable component design. SPI also led us to implement Linting (ESLint), Prettier, and better folder structure (`/components`, `/pages`, `/utils`) to improve readability and maintainability. Additionally, React Context and custom hooks like `useThemeToggle` helped decouple UI from logic, aligning with clean code practices.

## 3. Version Control Implementation

We implemented Git as our version control system and managed the project using GitHub, maintaining two separate repositories:

- The first repository holds the original implementation using HTML, CSS, and JavaScript.
- The second repository is the refactored version using React, Vite, React Router, and MUI.

All code was managed through branches with descriptive names (e.g., `feature/search`, `bugfix/theme-toggle`, `test/component-testing`). We adopted the Git Flow model for development, ensuring that all features were developed in isolation and merged through pull requests after thorough peer reviews. Commits followed conventional standards (e.g., `feat:`, `fix:`, `refactor:`) for clarity and traceability.

## 4. Lehman's Laws Justification in Our Project

Our project aligns with Lehman's Laws of Software Evolution in multiple ways.

- **Law of Continuing Change:** The application evolved from a static HTML site into a fully reactive, modern app, adapting to UI requirements and new libraries.
  - **Law of Increasing Complexity:** As features and routing logic grew, complexity increased, prompting us to refactor into hooks, contexts, and modular files.
  - **Law of Feedback System:** Regular sprint retrospectives, peer feedback, and CI checks guided our refinements.
  - **Law of Self-Regulation:** Based on sprint velocity, we adjusted our goals to ensure realistic and consistent progress.
- Our transition from the original codebase to a scalable SPA with routing, theming, and error boundaries illustrates these laws in practice.

## 5. Software Deployment Management

Though we haven't deployed the project on a live hosting platform yet, we structured it for easy deployment using Vite. The production build is created using Vite build, and deployment-ready files are generated in the folder. We have planned deployment on GitHub Pages or Netlify, which supports continuous deployment via GitHub integration. The application routing is managed via React Router, ensuring clean URLs and fallback <Not Found /> pages. The theming system is centralized using Theme Provider from MUI, ensuring consistent visuals across devices post-deployment.

## 6. Refactoring and Legacy Code Removal

A core goal of the project was to refactor legacy HTML/CSS/JS code into component based React code. Reusable components like <MovieCard>, <Navbar>, and <SearchBar> replaced repeated HTML blocks. All inline styling was removed in favor of Material UI styling, promoting consistency and responsiveness. The entire routing system, which was previously reliant on static navigation, was rebuilt using React Router v6. State management was abstracted using hooks and context instead of global variables. The result is a clean, maintainable codebase following React best practices and modern software architecture.

## 7. Unit Testing

Unit testing was implemented in the React codebase using Jest and React Testing Library. Each key component was tested in isolation to ensure functionality and prevent regressions. For instance, <MovieCard /> was tested for correct prop rendering, and <ThemeToggle /> was tested for dynamic state changes. Custom hooks like useLocalStorage were tested using mocks to simulate real-world behavior. These unit tests were integrated into our workflow and run automatically before any pull request is merged.

## 8. Automated Testing:

Automated testing in the *Horizon* project was achieved through the strategic use of unit testing, ensuring that individual components and functions behaved as expected under various conditions. We used Jest and React Testing Library to write and execute automated tests across the React application. These tests ran automatically during development and before any code was merged into the main branch, acting as a quality gate.

## 9. Exception Handling Concepts Applied

Proper exception handling was a critical part of the refactored application. API calls for movie data were wrapped in try...catch blocks, and failed responses triggered user-friendly error messages via MUI Snackbar components. We also created an Error Boundary in React to catch and gracefully handle unexpected runtime errors at the component level, thus preventing the entire app from crashing. Route mismatches are caught and directed to a custom <NotFound /> page. These practices ensure high application resilience.

## 10. Peer Reviews and Inspections

Our team followed a structured process where Pull Requests (PRs) were subjected to detailed manual code inspections before being merged into the main branch. Every PR was reviewed by at least one team member to ensure alignment with React best practices, proper use of Material UI, and consistent design and logic flow.

- **Abeeha Raza** led the **technical documentation and research** efforts, ensuring the project aligned with Software Construction principles. She also managed repositories, coordinated team workflows, and led sprint retrospectives.
- **Alina Faisal** was responsible for creating **professional presentation slides**, which visually captured the architecture, process model, testing strategy, and project evolution supporting both academic and stakeholder-facing communication.
- **Salman Rao** took ownership of the **unit testing framework**, ensuring key components and hooks were thoroughly tested through automated test scripts.
- **Haziq Zaman** led the **refactoring initiative**, transforming the original HTML/CSS/JS codebase into a modern React structure with reusable components and improved logic flow.

